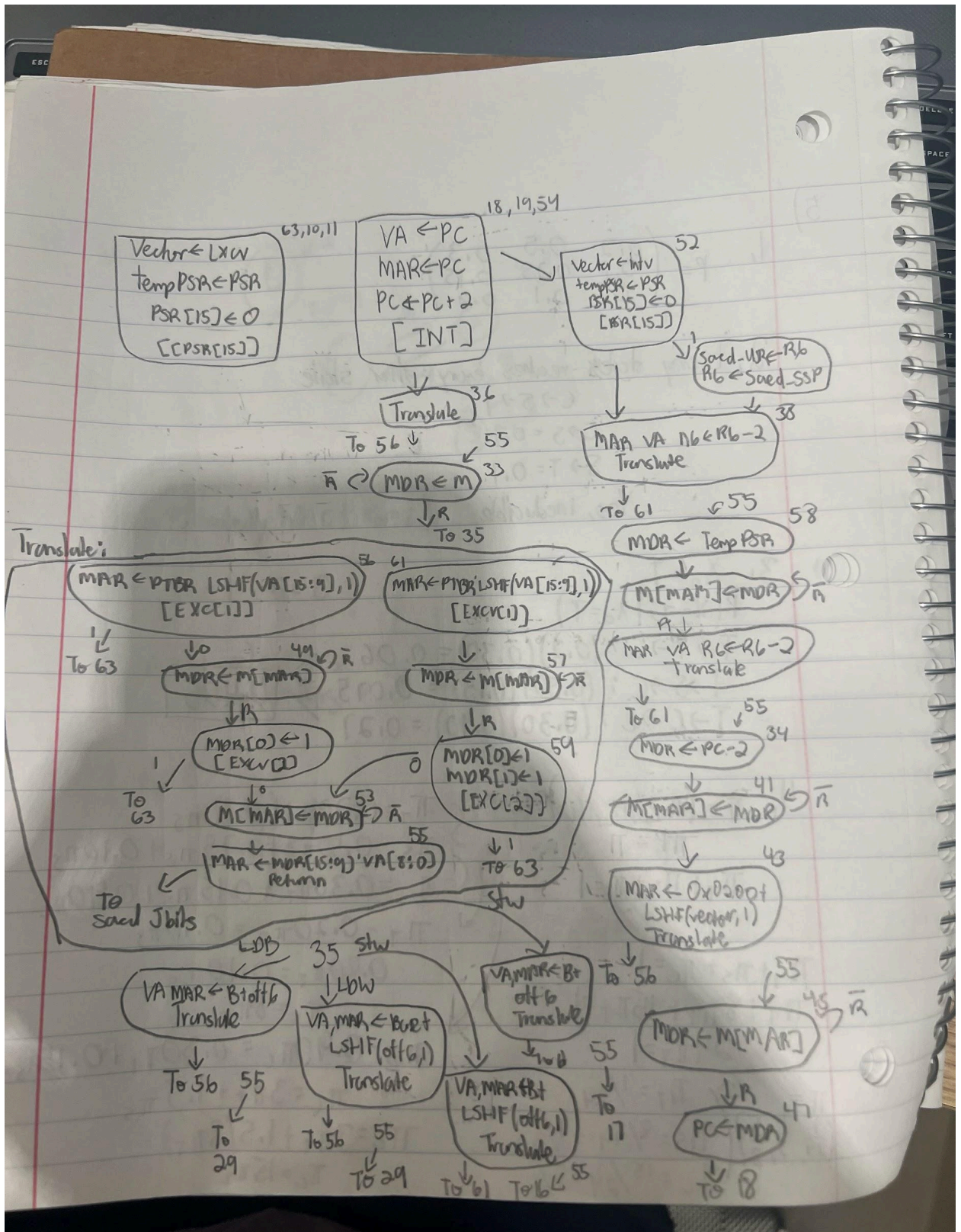
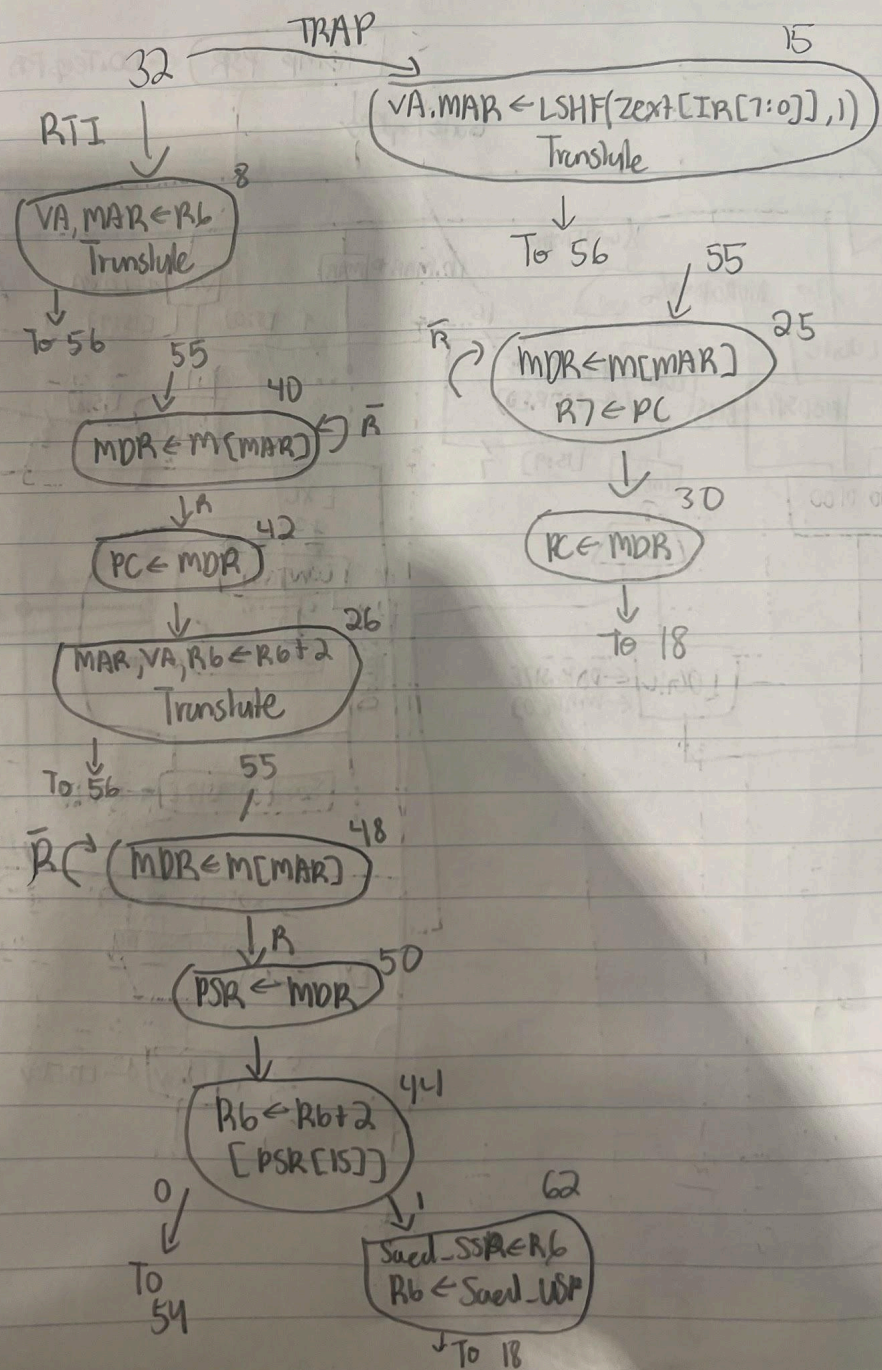






State Machine Addition:

To support virtual memory, additional states were added to manage the translation of virtual addresses into physical addresses. Specifically, states 56, 49, 51, 61, 57, 59, 53, and 55 were introduced for this translation process. In this design, the virtual address is stored in the MAR/VA registers. The VA register mirrors the MAR outside of the translation sequence. These states work together to create the Page Table Entry (PTE) for a given virtual address. They load the PTE into the MDR, update its reference bit, and, for a store operation, also its modified bit. The updated PTE is then written back to memory, and the physical address is computed using the PFN from the PTE along with the offset from the VA. Since load and store operations require different handling of the modified bit, the translation process starts in two separate states: state 56 for loads and state 61 for stores. These paths converge at state 53, where the updated PTE is written back to the page table. After that, the process continues through the other translation states until it is complete. All memory accesses are preceded by this translation sequence, marked by states labeled "TRANSLATE." During these states, the J bits of the next state are saved into a SavedJBits register. This allows the machine to return to the correct memory access state after the translation finishes, following state 55. Additionally, the DATA.SIZE control bit is kept in a SavedDATASIZE register, which is later used for checking unaligned access exceptions in state 56. Because the translation process alters the contents of the MDR, adjustments were needed to maintain correct behavior. In particular, in states 10, 11, 52, and 63, the PSR is no longer written directly to the MDR. Instead, it is saved in a TempPSR register and later restored to the MDR in state 58. Also, the operations in states 39 and 34 were swapped compared to Lab 4 to ensure proper handling of data during translation. The LC-3b control store contains 64 total states, which include the original Lab 3 states, Lab 4 interrupt and exception states, and the additional Lab 5 translation states.

State 56: Begins address translation for load operations by preparing to access the PTE.

State 61: Begins address translation for store operations, allowing the modified bit to be set.

State 49: Calculates the PTE address using the VA and PTBR and loads it into the MAR.

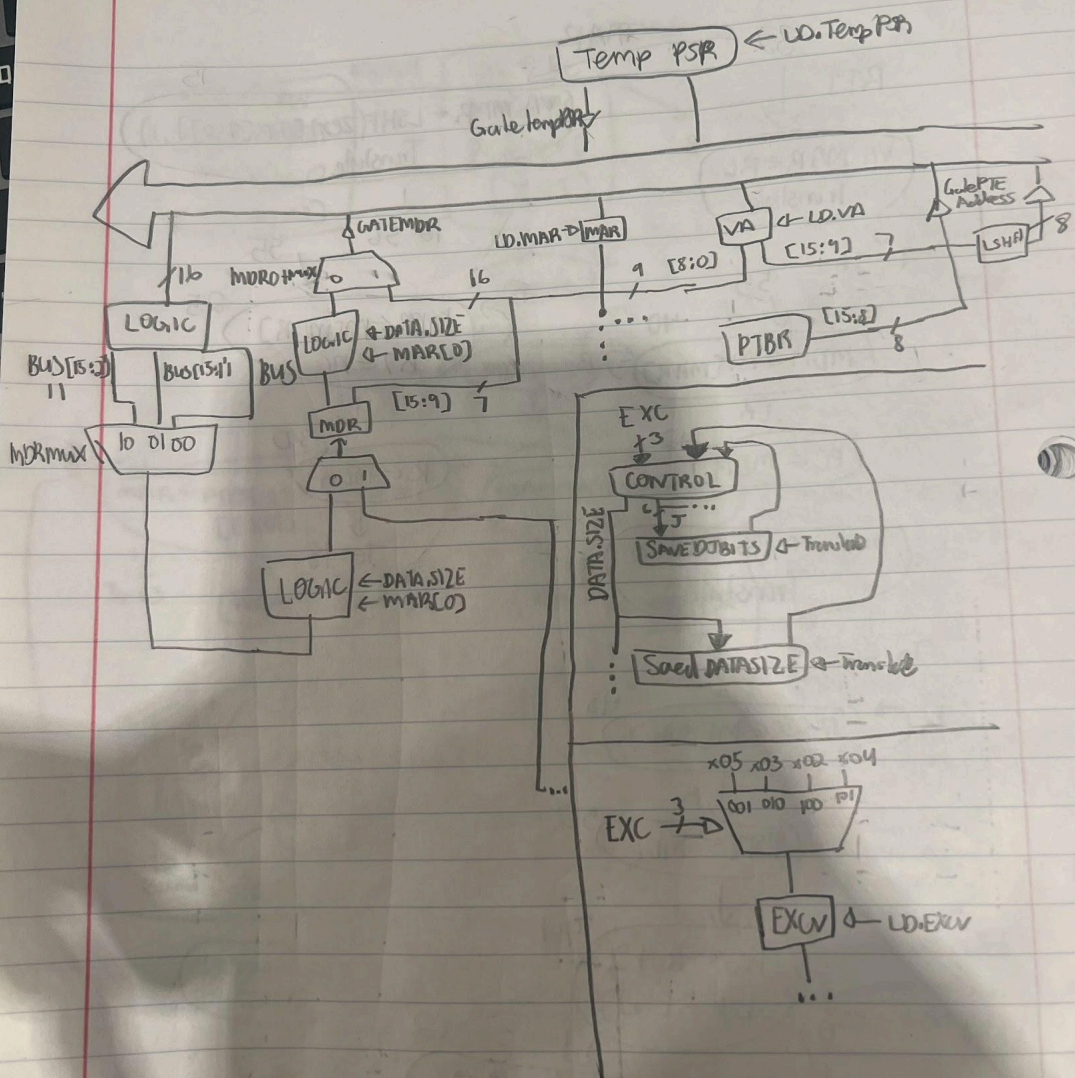
State 51: Reads the PTE from memory into the MDR.

State 57: Updates the PTE reference bit and prepares for modification if needed.

State 59: Sets the modified bit in the PTE for store operations.

State 53: Writes the updated PTE back to the page table in memory.

State 55: Constructs the final physical address from the PFN and VA offset and prepares to return to the original state.

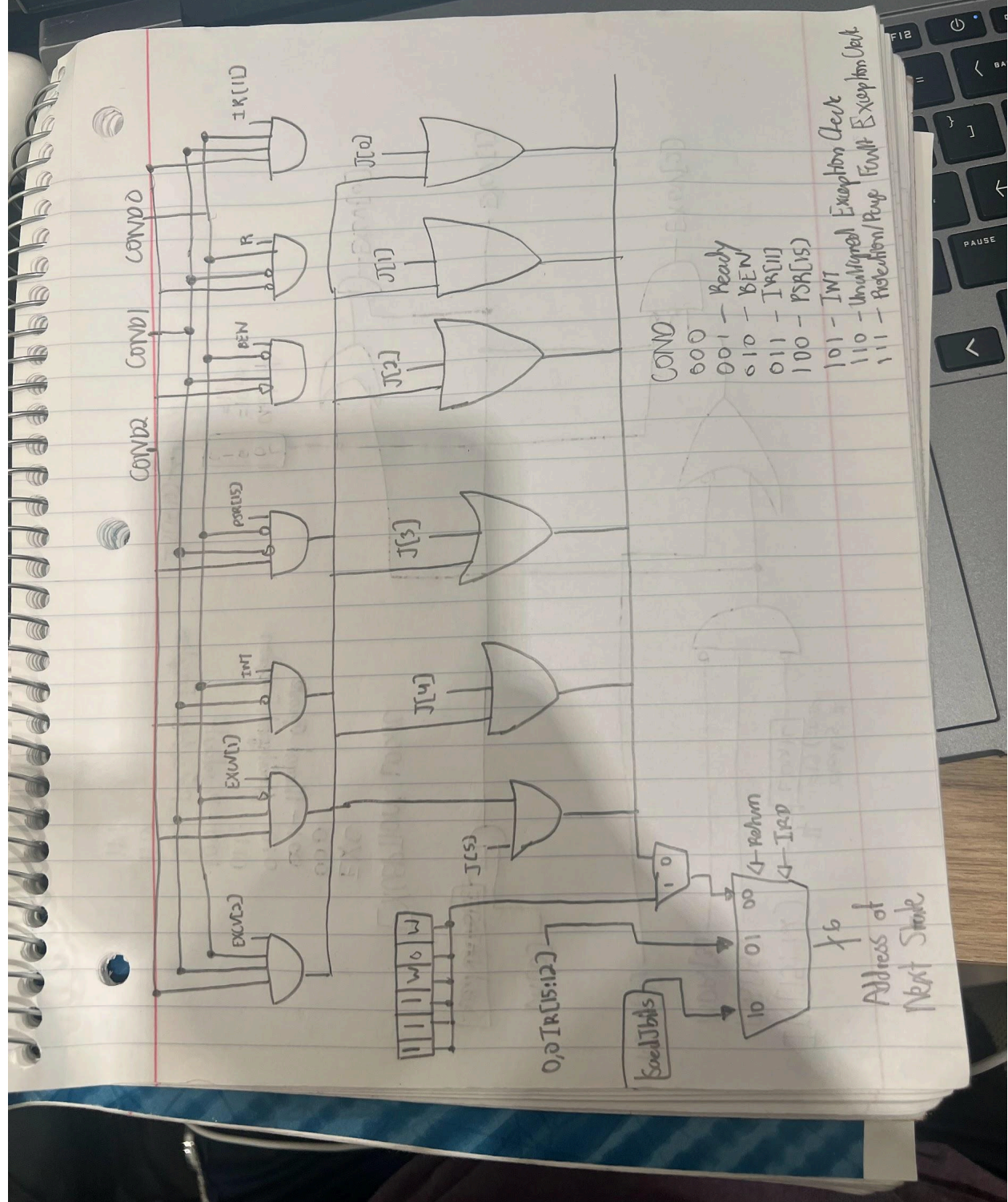


Control Path Modifications: To support virtual memory, several new components were added to the datapath. These include the TempPSR, VA, PTBR, SavedJBits, and SavedDATASIZE registers. Two new multiplexers, MDRInMUX and MDROutMUX, were also introduced to help with the address translation process. The GatePTEAddress control signal places a combination of VA[15:9] and PTBR[15:8] shifted left by one onto the bus. This forms the address of the corresponding Page Table Entry (PTE). The address is then loaded into the MAR, allowing the PTE to be fetched from memory into the MDR. After reading the PTE, its contents are updated based on the type of memory access. For load operations, the reference (valid) bit is set by OR-ing the MDR with x0001. For store operations, both the reference and modified bits are set by OR-ing with x0003. The updated PTE is then written back to the page table. Next, the physical address is constructed by combining the PFN from MDR[15:9] with the offset from VA[8:0]. This value is loaded into the MAR to complete the translation. Since this design introduces an additional exception type, the exception vector multiplexer has been expanded to include another input. The EXC signal has been extended to 3 bits to support the new exception logic, which is described later in the document.

New Control Signals:

These control signals were introduced to support virtual address translation and ensure correct control flow during memory accesses.

- TRANSLATE - Used to load the SavedJBits and SavedDATASize registers and in the microsequencer to set the next state number to 56 or 61 based on whether the W signal (R.W = 1) is asserted.
- RETURN - Used in the microsequencer to return to perform the memory access after address translation. If asserted it will use the SavedJBits register to get the next state.
- LD.VA - Used to load the VA register
- LD.TempPSR - Used to load the TempPSR register
- GatePTEAddress - Used to gate VA[15:9] and PTBR[15:8]<<1 (the address of the PTE) onto the bus
- GateTempPSR - Used to gate the TempPSR register onto the bus
- MDRInMux - Used to select an MDR input from the BUS: 00 = BUS, 01 = BUS[15:1]'1, 10 = BUS[15:2]'11
- MDROutMUX - Used to select the MDR output to be gated on the BUS: 0 = Output of logic block, 1 = MDR[15:9]'VA[8:0] (PA of given VA)



Microsequencer Changes: To handle virtual memory and more robust exception handling, they tweaked the microsequencer. They added a new COND value so it can spot different types of exceptions. Now, COND6 checks for unaligned access, and COND7 looks out for protection and page faults. Whenever the TRANSLATE signal kicks in, an extra multiplexer jumps in and reroutes things. It doesn't bother with the usual next-state logic—it pushes execution directly to state 56 if it's a load, or state 61 if it's a store, based on the R.W signal. If it doesn't see TRANSLATE, then the microsequencer sticks with its regular next-state logic using the old condition evaluation. They also expanded the big multiplexer that picks the next state. Now it's a three-input MUX, controlled by RETURN and IRD signals. That means the next state could come from the opcode (IRD), the normal control path, or straight from the SavedJBits register when returns are hit. What's neat about the SavedJBits register is that it lets the machine jump right back to the correct memory access state after sorting out address translation. So, with these updates, every memory access gets a proper virtual address translation first, execution stays on track, and exceptions are handled cleanly.

Exception Logic: EXC signals pop up whenever the datapath runs into trouble—things like unaligned memory accesses, protection violations, or page faults. The microsequencer relies on these signals to jump into the right exception handling state. Then there's the EXC value, which loads the EXCV register with the exception vector. That's what lets the system kick off a context switch and jump straight to the relevant exception handler.